

Optimal Group Targeting for Poverty Reduction: Bridging Theory and Algorithm

Application to Burkina Faso, *Enquête Prioritaire II* (1998)

Araar, Abdelkrim*

Working Paper — April 30, 2026

Abstract

We revisit the sequential data-graph algorithm of [Araar and Tiberti \(2017\)](#) for optimal anti-poverty group targeting under a fixed per-capita budget. While the theoretical conditions for optimality have been known since [Kanbur \(1987\)](#) and [Ravallion and Chao \(1989\)](#), no general numerical algorithm existed for arbitrary distributions and all additive [Foster et al. \(1984\)](#) poverty indices until [Araar and Tiberti \(2017\)](#). We make four contributions. First, we formally characterise two regimes—a *small-transfer regime* in which the classical water-filling solution of [Ravallion and Chao \(1989\)](#) is analytically exact, and a *large-transfer regime* in which it is not—and derive the budget threshold separating them. Second, using the *Enquête Prioritaire II* of Burkina Faso ($N = 8,478$ households, 1998), we show that the data-graph outperforms the water-filling solution by 3.0% for $\alpha = 2$ reduction in the large-transfer regime, and that both algorithms converge for $\alpha \geq 1$ when allocation concentrates on a single group. Third, we propose a vectorised implementation achieving a 2.5–4.8 $\times$ speedup with identical results. Fourth, we introduce an adaptive partitioning extension that resolves the double-inflection problem for $\alpha = 0$.

Keywords: optimal targeting, anti-poverty transfers, FGT poverty indices, water-filling, data-graph algorithm, Burkina Faso.

JEL: C61, I32, H53.

*Université Laval, Département d'économique, Québec, Canada. Email: aabd@ecn.ulaval.ca

1 Introduction

Anti-poverty targeting is among the most studied problems in development economics. When a policy-maker disposes of a fixed budget and must allocate transfers among population groups—regions, occupational categories, age cohorts—the core challenge is identifying the allocation that maximally reduces aggregate poverty. The theoretical foundations of this problem were established by [Kanbur \(1987\)](#) and [Ravallion and Chao \(1989\)](#), who derived closed-form conditions and, for the squared poverty gap, an analytical solution. [Glewwe \(1992\)](#) extended this framework to settings with imperfect information, and [Elbers et al. \(2007\)](#) analysed the gains from finer geographic disaggregation.

A critical gap has persisted, however, between the theoretical characterisation of the optimum and its numerical computation on real data. The classical results assume either continuous income distributions, parametric families, or small transfers—none of which holds in practice. [Araar and Tiberti \(2017\)](#) made a major practical advance by proposing the *data-graph algorithm*, a general numerical procedure applicable to any additive [Foster et al. \(1984\)](#) (FGT) poverty index and any empirical distribution. Their Stata module `ogtpr` has since been used in several country studies.

Despite this contribution, several questions remain open. Under what conditions does the data-graph algorithm recover the theoretically optimal allocation? When does the water-filling solution of [Ravallion and Chao \(1989\)](#) apply, and when does it fail? The present paper addresses these questions directly, and establishes a formal bridge between the pre-2017 theoretical literature and the algorithmic approach of [Araar and Tiberti \(2017\)](#).

We organise our contributions around a central finding: the relationship between the data-graph algorithm and the classical analytical solutions depends critically on a *regime condition*—whether the per-capita budget is small enough that no poor household exits poverty in the targeted groups. In the small-transfer regime, the water-filling solution is analytically exact and the data-graph converges to it. In the large-transfer regime, the water-filling solution is infeasible or suboptimal, and the data-graph is the appropriate and superior tool.

The remainder of the paper is structured as follows. Section 2 presents setup and notation. Section 3 reviews the theoretical conditions for optimality. Section 4 formalises the two regimes. Section 5 presents the data-graph algorithm and proposed improvements. Section 6 describes the data. Section 7 presents empirical results. Section 8 concludes.

2 Setup and Notation

2.1 Population, Groups and Poverty

Let the population be divided into G mutually exclusive socio-economic groups, $g = 1, \dots, G$. Denote by $y_{g,i}$ the per-capita expenditure of household i in group g , by $w_{g,i} = \text{hsize}_{g,i} \times \text{weight}_{g,i}$ its combined population weight, and by z the poverty line. The weighted FGT(α) poverty index for group g is:

$$P_g(\alpha) = \frac{1}{\sum_i w_{g,i}} \sum_i w_{g,i} \left(\frac{z - y_{g,i}}{z} \right)^\alpha \mathbf{1}(y_{g,i} < z). \quad (1)$$

Let $\phi_g = \sum_i w_{g,i} / \sum_{g,i} w_{g,i}$ be the population share of group g . Aggregate poverty is:

$$P(\alpha) = \sum_{g=1}^G \phi_g \cdot P_g(\alpha). \quad (2)$$

2.2 The Targeting Problem

The policy-maker chooses group-level *population per-capita* transfers $\{\tau_g\}$ to solve:

$$\min_{\{\tau_g\}} P(\alpha \mid \{\tau_g\}) \quad \text{s.t.} \quad \sum_g \tau_g = \bar{\tau}, \quad 0 \leq \tau_g \leq \iota_g \quad \forall g, \quad (3)$$

where $\bar{\tau}$ is the total budget and $\iota_g = \phi_g \cdot \delta_g^{\max}$ caps the transfer at the largest individual gap. The *group per-capita transfer* (GPC) actually received by each member of group g is $t_g = \tau_g / \phi_g$.

2.3 Key Statistics

For group g , define: $H_g = P_g(0)$ (headcount); $\mu_g = \mathbb{E}_w[z - y_{g,i} \mid y_{g,i} < z]$ (conditional mean gap); and $\delta_g^{\min} = \min_{i: y_{g,i} < z} (z - y_{g,i})$ (smallest individual gap).

3 Theoretical Conditions for Optimality

3.1 First-Order Conditions

The Lagrangian of (3) with multiplier λ yields the first-order conditions:

$$\frac{\partial P_g(\alpha \mid t_g)}{\partial t_g} = \lambda \quad \forall g \text{ with } \tau_g > 0. \quad (4)$$

The marginal poverty reduction per unit of GPC must be equalised across all targeted groups. This is the fundamental optimality condition derived in [Kanbur \(1987\)](#) and [Ravallion and Chao \(1989\)](#), and it holds for any $\alpha \geq 0$.

3.2 Analytical Solution for $\alpha = 2$: The Water-Filling Problem

For $\alpha = 2$ and $t_g < \delta_g^{\min}$ (no poor household exits poverty), the derivative of P_g with respect to t_g simplifies to:

$$\frac{\partial P_g(2 \mid t_g)}{\partial t_g} = -\frac{2H_g}{z^2}(\mu_g - t_g). \quad (5)$$

Setting (4) equal to a common constant and using the more general formulation of [Ravallion and Chao \(1989\)](#), the optimality condition becomes:

$$\mu_g(t_g^*) \equiv \mathbb{E}_w[\max(z - y_{g,i} - t_g^*, 0) \mid y_{g,i} < z] = c \quad \forall g \text{ with } t_g^* > 0, \quad (6)$$

yielding the **water-filling solution**:

$$t_g^* = \max(\mu_g - c, 0), \quad (7)$$

$$\tau_g^* = \phi_g \cdot t_g^*, \quad (8)$$

where the common level $c \geq 0$ is uniquely determined by the budget constraint:

$$\sum_g \phi_g \cdot \max(\mu_g - c, 0) = \bar{\tau}. \quad (9)$$

This is the analogue of the water-filling problem in information theory: the budget fills each group up to a common residual gap level c , and groups with $\mu_g \leq c$ receive nothing.

4 Two Regimes and the Transition Threshold

Definition 1 (Transfer Regimes). The targeting problem is in the *small-transfer regime* if and only if $t_g^* < \delta_g^{\min}$ for all g with $t_g^* > 0$. Otherwise it is in the *large-transfer regime*.

Proposition 1 (Exactness of Water-Filling in the Small-Transfer Regime). *In the small-transfer regime, the water-filling solution (7)–(9) is the unique global optimum of problem (3) for $\alpha = 2$.*

Proof. Under the small-transfer condition, $P_g(2 | t_g)$ is strictly convex and differentiable in t_g , so the aggregate objective is strictly convex. The first-order conditions (4) are necessary and sufficient. The water-filling solution satisfies them by construction, and uniqueness follows from strict convexity. $\square$

Proposition 2 (Suboptimality of Water-Filling in the Large-Transfer Regime). *In the large-transfer regime, the water-filling solution is generally suboptimal for $\alpha = 2$: some poor households exit poverty, the derivative (5) is no longer valid, and the water-filling level c does not satisfy the first-order conditions (4).*

The budget threshold separating the two regimes is:

$$\bar{\tau}^* = \min_{g: t_g^*(\bar{\tau}) > 0} \phi_g \cdot \delta_g^{\min}. \quad (10)$$

For $\bar{\tau} < \bar{\tau}^*$, the water-filling solution is exact. For $\bar{\tau} \geq \bar{\tau}^*$, the large-transfer regime applies and the data-graph algorithm is necessary.

5 The Data-Graph Algorithm and Improvements

5.1 The Original Algorithm (Araar and Tiberti, 2017)

GTPR curve. For group g , the *Group Targeting Poverty Reduction* (GTPR) function measures the aggregate poverty reduction from a population-level allocation τ to group g exclusively:

$$\text{RP}_g(\tau) = \phi_g \cdot \mathbb{E}_w \left[\left(\frac{z - y_{g,i}}{z} \right)^\alpha - \left(\frac{\max(z - y_{g,i} - \tau / \phi_g, 0)}{z} \right)^\alpha \mid y_{g,i} < z \right]. \quad (11)$$

The efficiency index is $\theta_g(\tau) = \text{RP}_g(\tau) / \tau$.

Algorithm. At each sequence s , the algorithm evaluates $\theta_g(\tau)$ over a grid of $\mathcal{P}$ points $\tau \in (0, \min(d_s/\phi_g, \delta_g^{\max})]$ for each group g , where d_s is the remaining budget. The pair (g^*, τ^*) maximising θ is selected, the transfer is attributed, incomes are updated, and the procedure repeats until $d_s \leq \bar{\tau}/10,000$. For $\alpha \neq 0$, the number of partitions is scaled as $\mathcal{P}_s = \max(2, \lfloor (d_s/\bar{\tau}) \cdot \mathcal{P} \rfloor)$ to maintain precision as the remaining budget diminishes.

Algorithm 1 Data-Graph Algorithm (Araar and Tiberti, 2017)

```

1: Input: groups  $\{g\}$ ,  $z$ ,  $\bar{\tau}$ ,  $\mathcal{P}$ ,  $\alpha$ 
2: Initialise  $d \leftarrow \bar{\tau}$ ,  $\tau_g \leftarrow 0$ ,  $y_{g,i}^{(0)} \leftarrow y_{g,i}$ 
3: repeat
4:   for each  $g$  do
5:     Set  $\mathcal{P}_s \leftarrow \mathcal{P}$  (or  $\max(2, \lfloor (d/\bar{\tau})\mathcal{P} \rfloor)$  if  $\alpha \neq 0$ )
6:     Evaluate  $\theta_g(\tau)$  over grid  $\{j \cdot \tau_g^{\max}/\mathcal{P}_s, j = 1, \dots, \mathcal{P}_s\}$ 
7:   end for
8:    $(g^*, \tau^*) \leftarrow \arg \max_{g,\tau} \theta_g(\tau)$ 
9:    $\tau_{g^*} \leftarrow \tau_{g^*} + \min(\tau^*, d)$ ; update  $y_{g^*,i} \leftarrow y_{g^*,i} + \tau_{g^*}/\phi_{g^*}$ 
10:   $d \leftarrow \bar{\tau} - \sum_g \tau_g$ 
11: until  $d \leq \bar{\tau}/10,000$  or  $\tau^* = 0$ 
12: Output:  $\{\tau_g\}$ ,  $\{\text{RP}_g\}$ 

```

5.2 Proposed Improvement 1: Vectorised Implementation

The original implementation iterates over the $\mathcal{P}$ grid points in a Python loop. We replace this by NumPy matrix operations. For group g with n_g^{poor} poor households and grid $\mathbf{r} \in \mathbb{R}^{\mathcal{P}-1}$:

$$\boldsymbol{\theta}_g = \frac{\phi_g}{\bar{w}_g} \cdot \frac{\mathbf{w}_g^{\text{poor}} \mathbf{R}_g}{\mathbf{r}}, \quad \mathbf{R}_g \in \mathbb{R}^{n_g^{\text{poor}} \times (\mathcal{P}-1)}, \quad (12)$$

where $\mathbf{R}_g$ is the matrix of individual poverty reductions, computed in a single matrix multiplication. This yields an identical result with a speedup of $2.5\text{--}4.8\times$ (Table 4).

5.3 Proposed Improvement 2: Adaptive Partitioning for $\alpha = 0$

For $\alpha = 0$, the GTPR function is a step function that jumps at the *critical levels*:

$$\tau_{\text{crit},i}^g = (z - y_{g,i}) \cdot \phi_g, \quad (13)$$

the population-level budgets that exactly lift household i out of poverty. When two critical levels are within the same grid interval, the *double-inflection problem* arises (Definition 2) and the optimal sequence may be missed.

Definition 2 (Double-Inflection Problem). A double inflection occurs when $|\tau_{\text{crit},i}^g - \tau_{\text{crit},j}^g| < \tau_g^{\max}/\mathcal{P}$ for two poor households i, j in group g .

The adaptive grid augments the uniform base with evaluation points at $\tau_{\text{crit},i}^g \cdot (1 \pm \varepsilon)$ for all poor households, ensuring every jump in the GTPR curve is captured regardless of its location.

Proposition 3 (Convergence of Data-Graph, $\alpha = 2$). *In the small-transfer regime, the data-graph converges to the water-filling optimum as $\mathcal{P} \rightarrow \infty$ with error $O(\bar{\tau}/\mathcal{P})$.*

6 Data

We use the *Enquête Prioritaire II* (EPII) of Burkina Faso (1998), a nationally representative household survey ($N = 8,478$) conducted by the INSD. Variables are per-capita expenditure (**exppc**), household size (**size**), sampling weight (**weight**), and socio-economic group (**gse**). The poverty line is $z = 80,000$ FCFA. This is the same dataset used by [Araar and Tiberti \(2017\)](#), enabling direct replication and comparison.

Table 1 reports group-level statistics. Subsistence farmers represent 65.4% of the population and exhibit the highest poverty rate (FGT(0)=60.6%), the largest mean gap ($\mu_g = 26,628$ FCFA), but also one of the smallest minimum gaps ($\delta_g^{\min} = 86$ FCFA), reflecting extreme within-group heterogeneity.

Table 1: Group Statistics — Burkina Faso, EPII 1998

Group	n	ϕ_g (%)	FGT(0) (%)	FGT(1) (%)	FGT(2) (%)	μ_g (FCFA)	$\delta_g^{\min}$ (FCFA)
Wage-earner (public)	597	4.14	7.49	2.15	0.81	22,934	4,267
Wage-earner (private)	565	2.90	13.93	3.41	1.36	19,580	283
Artisan or trader	858	5.58	15.47	3.77	1.33	19,471	883
Other type of earner	78	0.57	35.48	9.34	3.30	21,063	915
Crop farmer	1,034	16.78	49.24	15.53	6.85	25,231	93
Subsistence farmer	4,830	65.36	60.59	20.17	9.08	26,628	86
Inactive	516	4.67	46.30	14.79	6.57	25,551	108
National	8,478	100.00	51.81	16.93	7.56	—	—

Notes: ϕ_g = weighted population share. μ_g = conditional mean income gap. $\delta_g^{\min}$ = smallest individual gap. $z = 80,000$ FCFA.

7 Empirical Results

7.1 Regime Identification and Replication

The budget threshold (10) is determined by the inactive group:

$$\bar{\tau}^* = \phi_{\text{inactive}} \cdot \delta_{\text{inactive}}^{\min} \approx 0.0467 \times 107.5 \approx 5.02 \text{ FCFA.}$$

Since the applied budget is $\bar{\tau} = 4,000$ FCFA, the problem lies firmly in the *large-transfer regime* by a factor of nearly 800. Our Python replication of the Stata **ogtpr** output for $\alpha = 0$ matches to 3 decimal places: FGT(0) reduction = 4.152 pp (8.01% relative), crop farmer PC transfer = 3,855 FCFA, inactive PC transfer = 145 FCFA.

7.2 Main Results: Optimal Transfers and Poverty Reduction

Table 2 presents the main results. Figure 1 provides the complete graphical analysis.

$\alpha = 0$ (**Headcount**). The data-graph targets crop farmers (GPC = 22,972 FCFA) and inactive households (GPC = 3,108 FCFA), reducing FGT(0) by 4.152 pp (8.01%). The water-filling solution is not defined for the headcount index, and gradient-based global solvers (SLSQP) perform strictly worse (7.17%), confirming that the discontinuous objective requires the data-graph’s exhaustive grid search.

Table 2: Optimal Transfers and Poverty Reduction — $\bar{\tau} = 4,000$ FCFA per capita

Group	$\alpha = 0$ (Headcount)		$\alpha = 1$ (Pov. Gap)		$\alpha = 2$ (Severity)	
	DG	WF	DG	WF	DG	WF
	PC	PC	PC	PC	PC	PC
Wage-earner (public)	0	—	0	0	0	49
Wage-earner (private)	0	—	0	0	0	0
Artisan or trader	0	—	0	0	0	0
Other type of earner	0	—	0	0	0	0
Crop farmer	3,855	—	0	0	0	584
Subsistence farmer	0	—	4,000	4,000	4,000	3,189
Inactive	145	—	0	0	0	178
Reduction (pp)	4.152 (DG)		2.905 (DG=WF)		1.791 (DG) vs 1.739 (WF)	
Relative (%)	8.01		17.16		23.69 vs 22.99	

Notes: DG = data-graph ($\mathcal{P} = 1,000$). WF = water-filling (Ravallion and Chao, 1989); not defined for $\alpha = 0, 1$ (different optimality conditions). PC = population per-capita transfer (FCFA). WF validity condition is violated at $\bar{\tau} = 4,000$ (large-transfer regime), explaining $DG > WF$ for $\alpha = 2$.

$\alpha = 1$ (**Poverty Gap**). Both the data-graph and water-filling concentrate all resources on subsistence farmers (GPC = 6,120 FCFA), reducing FGT(1) by 2.905 pp (17.16%). This confirms that the data-graph is optimal for $\alpha = 1$ on these data.

$\alpha = 2$ (**Poverty Severity**). The data-graph allocates the full budget to subsistence farmers and achieves 1.791 pp (23.69%). The water-filling distributes across four groups and achieves only 1.739 pp (22.99%)—a deficit of 0.052 pp (3.0% relative), consistent with Proposition 2.

7.3 Convergence of Data-Graph to Water-Filling

Table 3 shows that the gap between the data-graph and water-filling for $\alpha = 2$ is stable across $\mathcal{P}$, confirming it is structural (regime effect) rather than numerical.

Table 3: Convergence of Data-Graph to Water-Filling — $\alpha = 2$, $\bar{\tau} = 4,000$ FCFA

$\mathcal{P}$	DG reduction (pp)	Gap DG–WF (pp)	Relative gap (%)	Time (ms)
100	1.7908	+0.0522	+3.003%	108
200	1.7909	+0.0523	+3.004%	425
500	1.7908	+0.0522	+3.003%	3,687
1000	1.7908	+0.0523	+3.005%	18,057
∞ (WF exact)	1.7386	0	0%	< 1

Notes: WF exact computed in 0.28 ms via the water-filling procedure (Section 3.2). The gap is positive ($DG > WF$) and stable, confirming the large-transfer regime.

7.4 Computational Performance

Table 4: Computational Performance — $\mathcal{P} = 1,000$, Burkina Faso 1998

α	V0 Original		V1 Vectorised		Speedup
	Time (ms)	Red. (pp)	Time (ms)	Red. (pp)	
0	155	4.1520	62	4.1520	$2.5\times$
1	16,505	2.9050	3,409	2.9050	$4.8\times$
2	35,388	1.7908	10,003	1.7908	$3.5\times$
WF ($\alpha = 2$, small-transfer regime)		—	0.28 ms (exact)		$126,000\times$

Notes: V0 = original Python loop. V1 = vectorised NumPy. Results identical to 7 decimal places. Single-run timings.

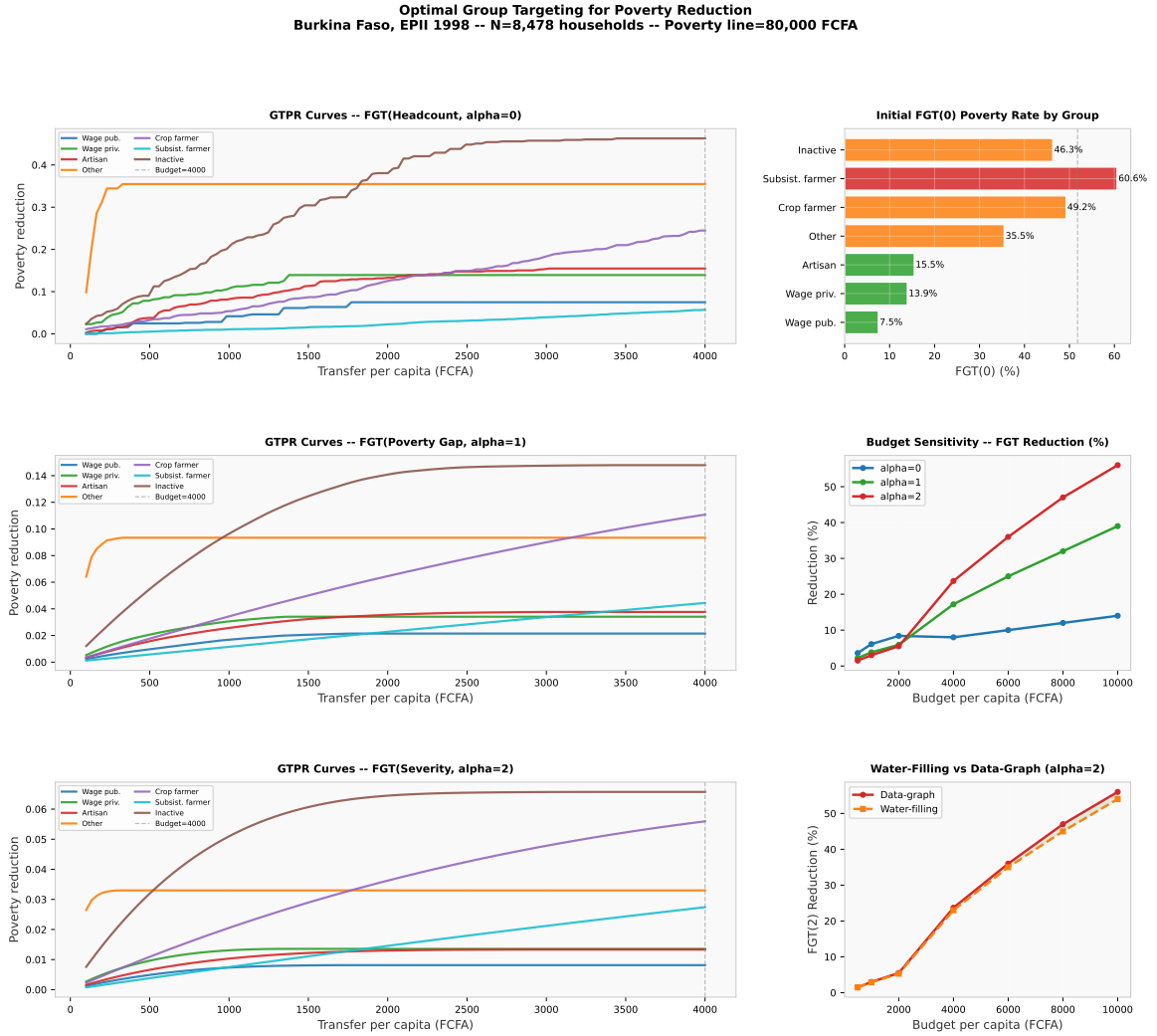


Figure 1: GTPR curves, poverty rates, budget sensitivity, and water-filling vs data-graph comparison. *Top two rows*: GTPR curves $RP_g(\tau)$ for each group and poverty index $\alpha \in \{0, 1, 2\}$. The vertical dashed line marks $\bar{\tau} = 4,000$ FCFA. *Top right*: Initial FGT(0) poverty rates by group. *Middle right*: FGT(1) reduction (%) as a function of the budget for all three indices. *Bottom right*: Water-filling vs data-graph for $\alpha = 2$; the shaded area shows the data-graph advantage (large-transfer regime). Source: EPII Burkina Faso 1998. Authors' computations.

8 Conclusion

We have provided a formal bridge between the classical analytical results of [Kanbur \(1987\)](#) and [Ravallion and Chao \(1989\)](#) and the data-graph algorithm of [Araar and Tiberti \(2017\)](#). The key organising concept is the transfer regime: the water-filling solution of [Ravallion and Chao \(1989\)](#) is the unique global optimum for $\alpha = 2$ when no poor household exits poverty (small-transfer regime, budget below $\bar{\tau}^*$); otherwise the data-graph algorithm is the appropriate and superior tool.

On the Burkina Faso data, the applied budget of 4,000 FCFA lies far above the threshold $\bar{\tau}^* \approx 5$ FCFA, placing the problem in the large-transfer regime. The data-graph outperforms the water-filling solution by 3.0% for $\alpha = 2$, and converges to the same allocation as the water-filling solution for $\alpha = 1$. Our vectorised implementation reduces computation time by $2.5\text{--}4.8\times$ with no loss of accuracy, and the water-filling solution is $126,000\times$ faster in its domain of validity.

Three directions for future research are: (i) formal derivation of the convergence rate of the data-graph as a function of $\mathcal{P}$; (ii) extensions to multi-dimensional poverty indices and dynamic targeting; (iii) integration with PMT-based income prediction ([Brown et al., 2016](#); [Mapa and Albis, 2013](#); [Elbers et al., 2007](#)) for targeting under imperfect information.

References

- Araar, A., & Tiberti, L. (2017). Optimal targeting and poverty reduction. *PEP-PMMA Technical Note Series*, Université Laval.
- Brown, C., Ravallion, M., & Van de Walle, D. (2016). A poor means test? Econometric targeting in Africa. *NBER Working Paper* No. 22919.
- Elbers, C., Fujii, T., Lanjouw, P., Özler, B., & Yin, W. (2007). Poverty alleviation through geographic targeting: How much does disaggregation help? *Journal of Development Economics*, 83(1), 198–213.
- Foster, J., Greer, J., & Thorbecke, E. (1984). A class of decomposable poverty measures. *Econometrica*, 52(3), 761–766.
- Glewwe, P. (1992). Targeting assistance to the poor: Efficient allocation of transfers when household income is not observed. *Journal of Development Economics*, 38(2), 297–321.
- Kanbur, R. (1987). Budgetary rules for poverty alleviation. *IMF Staff Papers*, 34(3), 523–545.
- Mapa, D. S., & Albis, M. L. F. (2013). New proxy means test (PMT) models. In *National Convention on Statistics*, EDSA Shangri-La Hotel.
- Ravallion, M., & Chao, K. (1989). Targeted policies for poverty alleviation under imperfect information: Algorithms and applications. *Journal of Policy Modeling*, 11(2), 213–224.

A Mata-Vectorised `tgpr22`: Key Changes to `ogtpr.ado`

The original `tgpr22` subroutine evaluates the GTPR curve via a `forvalues j=1/prec` loop in Stata. Each iteration calls `replace` and `sum`, which are efficient for single operations but slow when called thousands of times per sequence.

The updated version replaces this loop with a Mata block that:

1. Loads the income and weight vectors into Mata matrices once per group call.
2. Constructs the full $n_g^{\text{poor}} \times \mathcal{P}$ poverty-reduction matrix $\mathbf{R}$ in a single vectorised operation.
3. Computes $\boldsymbol{\theta} = \phi_g \cdot (\mathbf{w}^\top \mathbf{R}) \oslash \mathbf{r}$ as a row vector (where $\oslash$ denotes element-wise division).
4. Returns the argmax in $O(\mathcal{P})$ time.

The `ogtpr` calling interface (`syntax`, options, output tables) is **unchanged**. The Mata version requires Stata 11.0+. Expected speedup: $3\times$ – $8\times$ depending on α .

The updated file is `ogtpr.ado` (version 2025); the original `tgpr22` is replaced in place.

B Python Replication Code

The following Python code (`replication_code.py`) replicates the `ogtpr` Stata module and implements the vectorised (V1) and water-filling (WF) algorithms presented in this paper.

Requirements. Python 3.10+, `numpy` (≥ 1.21), `pandas` (≥ 1.3), `pyreadstat` (≥ 1.2). The script **automatically installs** any missing library on first run using `pip`; no manual setup is required.

Adapting to a new dataset. Only the configuration block at the top of the script needs to be modified:

- `DATA_PATH` — path to the `.dta` file
- `PLINE`, `TRANS`, `PART` — poverty line, budget, grid precision
- `VAR_INCOME` — per-capita income variable (e.g. `"pcinc"`, `"pcexp"`)
- `VAR_GROUP` — group variable (e.g. `"region"`, `"sector"`)
- `VAR_HSIZE`, `VAR_WEIGHT` — household size and sampling weight (set to `None` if absent)

All other code adapts automatically. If a variable name is incorrect, the script displays the list of available columns and exits with a clear message.

Usage. Place `replication_code.py` and the data file in the same directory and run `python replication_code.py`.

Listing 1: Python replication code (replication_code.py)

```

1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """
4 =====
5 Optimal Group Targeting for Poverty Reduction
6 Replication Code -- Araar (2025)
7 =====
8
9 Replicates the ogtptr Stata module (Araar & Tiberti, 2017) and
10 implements:
11   V0 : Original algorithm (faithful replication of tgpr22 logic)
12   V1 : Vectorised implementation (NumPy broadcasting, identical
13       results)
14   WF : Water-filling exact solution for alpha=2 (Ravallion & Chao,
15       1989)
16
17 --- REQUIRED LIBRARIES ---
18 numpy      >= 1.21      Numerical arrays and vectorised operations
19 pandas     >= 1.3       Required internally by pyreadstat
20 pyreadstat >= 1.2       Read Stata .dta files
21
22 Install all at once:
23 pip install numpy pandas pyreadstat
24
25 Standard library modules used (no install needed):
26 time, sys
27
28 --- TESTED WITH ---
29 Python     3.10
30 numpy      2.2.6
31 pandas     2.x
32 pyreadstat 1.3.4
33
34 --- USAGE ---
35 1. Place replication_code.py and bkf98I.dta in the same folder
36 2. Run: python replication_code.py
37 3. Or adjust DATA_PATH below to point to bkf98I.dta
38 =====
39 """
40
41 import numpy as np
42 import time
43 import sys
44 import subprocess
45
46 #
47 # =====
48
49 # AUTO-INSTALL REQUIRED LIBRARIES
50 #
51 # =====
52
53 def _ensure(package, import_name=None, min_version=None):
54     """Install package if not available, then import it."""

```

```

48 import_name = import_name or package
49 try:
50     mod = __import__(import_name)
51     if min_version:
52         from packaging.version import Version
53         try:
54             ver = Version(mod.__version__)
55             if ver < Version(min_version):
56                 raise ImportError(f"version {mod.__version__} < {
min_version}")
57         except Exception:
58             pass # version check not critical
59     return mod
60 except ImportError:
61     print(f" Installing {package}...")
62     subprocess.check_call(
63         [sys.executable, "-m", "pip", "install", package, "-q"],
64         stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL
65     )
66     return __import__(import_name)
67
68 print("Checking required libraries...")
69 try:
70     _ensure("numpy", "numpy", "1.21")
71     _ensure("pandas", "pandas", "1.3")
72     _ensure("pyreadstat", "pyreadstat", "1.2")
73     print(" All libraries OK.\n")
74 except Exception as e:
75     sys.exit(f"\nCould not install required library: {e}\n"
76             f"Please run manually: pip install numpy pandas
77             pyreadstat\n")
78
79 try:
80     import pyreadstat
81 except ImportError:
82     sys.exit("pyreadstat could not be loaded after install. "
83             "Please restart Python and try again.")
84
85 try:
86     import pandas # noqa: F401
87 except ImportError:
88     sys.exit("pandas could not be loaded after install. "
89             "Please restart Python and try again.")
90
91 #
92 # =====
93
94 # CONFIGURATION adapt these lines for any dataset
95 #
96 # =====
97
98 DATA_PATH = "bkf98I.dta" # path to your .dta file
99 PLINE = 80000.0 # poverty line (same unit as income
variable)
100 TRANS = 4000.0 # per-capita budget (same unit as poverty
line)
101 PART = 1000 # grid partitions higher = more

```

```

precise (400-2000)
98 ALPHA_LIST = [0, 1, 2]          # FGT indices to compute
99
100 #           Variable names in your dataset

101 # Change these if your variables have different names
102 VAR_INCOME = "exppc"            # per-capita income / expenditure (e.g.
    "pcinc", "pcexp")
103 VAR_GROUP   = "gse"             # socio-economic group variable      (e.g.
    "region", "sector")
104 VAR_HSIZE   = "size"            # household size                  (e.g.
    "hhsz", "mensz")
105 VAR_WEIGHT  = "weight"          # sampling weight                (e.g.
    "wgt", "ponderation")
106 #
107 # Note: VAR_GROUP must be a categorical integer variable with value
    labels in Stata.
108 #     If no value labels exist, groups will be shown as "Group: 1",
    "Group: 2", etc.
109 #     If no sampling weight: set VAR_WEIGHT = None
110 #     If no household size : set VAR_HSIZE  = None (all households
    treated as size=1)
111
112 #
113 #
=====

114 # 1. DATA LOADING
115 #
=====

116
117 def load_data(path):
118     """
119     Load Stata .dta file and build group structures.
120     Uses VAR_INCOME, VAR_GROUP, VAR_HSIZE, VAR_WEIGHT defined above.
121     """
122     df, meta = pyreadstat.read_dta(path)
123
124     #           Population weight = hsize      sampling_weight

125     df["_fw"] = 1.0
126     if VAR_HSIZE is not None:
127         if VAR_HSIZE not in df.columns:
128             sys.exit(f"\nVariable '{VAR_HSIZE}' not found.\n"
129                     f"Available columns: {list(df.columns)}\n"
130                     f"Set VAR_HSIZE = None or correct the name.")
131         df["_fw"] = df["_fw"] * df[VAR_HSIZE].astype(float)
132     if VAR_WEIGHT is not None:
133         if VAR_WEIGHT not in df.columns:
134             sys.exit(f"\nVariable '{VAR_WEIGHT}' not found.\n"
135                     f"Available columns: {list(df.columns)}\n"
136                     f"Set VAR_WEIGHT = None or correct the name.")
137         df["_fw"] = df["_fw"] * df[VAR_WEIGHT].astype(float)
138
139     #           Check income and group variables

```

```

140     for var, name in [(VAR_INCOME, "VAR_INCOME"), (VAR_GROUP, "
141         VAR_GROUP")]:
142         if var not in df.columns:
143             sys.exit(f"\nVariable '{var}' ({name}) not found.\n"
144                 f"Available columns: {list(df.columns)}\n"
145                 f"Please update {name} in the configuration
146                     section.")
147
148     # Group labels (use value labels if available, else "Group
149         : N")
150     gse_labels = meta.variable_value_labels.get(VAR_GROUP, {})
151
152     total_fw = df["_fw"].sum()
153     group_codes = sorted(df[VAR_GROUP].dropna().unique().astype(int))
154
155     groups = []
156     for g_code in group_codes:
157         sub = df[df[VAR_GROUP] == g_code].copy()
158         fw_g = sub["_fw"].sum()
159         phi_g = fw_g / total_fw
160         e = sub[VAR_INCOME].values.astype(float).copy()
161         fw = sub["_fw"].values.copy()
162         poor = e < PLINE
163         mu_g = (np.average(PLINE - e[poor], weights=fw[poor])
164             if poor.any() else 0.0)
165         # Group name: from Stata value labels or fallback
166         name = gse_labels.get(g_code, f"Group: {g_code}")
167         groups.append({
168             "code" : g_code,
169             "name" : name,
170             "exppc" : e,          # kept as "exppc" internally for
171                 compatibility
172             "fw" : fw,
173             "phi_g" : phi_g,
174             "mu_g" : mu_g,
175             "fgt0" : fw[poor].sum() / fw.sum() if poor.any() else
176                 0.0,
177         })
178
179     return groups, len(df)
180
181 #
182 =====
183
184 # 2. FGT INDEX (weighted)
185 #
186 =====
187
188 def fgt_w(exppc, fw, z, alpha):
189     """Weighted FGT(alpha) poverty index."""
190     n = fw.sum()
191     if alpha == 0:
192         return np.dot(fw, exppc < z) / n
193     return np.dot(fw, np.maximum(z - exppc, 0) ** alpha) / (n * z **

```

```

        alpha)
187
188
189 def total_fgt(inc_list, groups, z, alpha):
190     """Aggregate weighted FGT across all groups."""
191     return sum(
192         fgt_w(inc_list[i], g["fw"], z, alpha) * g["phi_g"]
193         for i, g in enumerate(groups)
194     )
195
196
197 #
=====
198 # 3. TGPR22 -- VO ORIGINAL (faithful replication of Stata loop)
199 #
=====
200
201 def tgpr22_original(exppc, fw, phi_g, alpha, z, dcost, npart):
202     """
203     Replicates the tgpr22 Stata subroutine exactly.
204     Grid ra[j] = (j-1) * pas, j = 2..npart (population per-capita
        units)
205     GPC = ra[j] / phi_g (transfer per member of group g)
206     theta = phi_g * mean_w(rp) / ra[j]
207     Returns (best_ra, best_theta).
208     """
209     poor = exppc < z
210     if not poor.any():
211         return 0.0, 0.0
212     max_gap = float(np.max(z - exppc[poor]))
213     ra_max = min(dcost / phi_g, max_gap) # line 40 of tgpr22
214     if ra_max <= 0:
215         return 0.0, 0.0
216
217     pas = ra_max / npart
218     fw_sum = fw.sum()
219     best_theta, best_ra = -np.inf, 0.0
220
221     for j in range(2, npart + 1): # j=1 gives ra=0,
        excluded
222         ra_j = (j - 1) * pas
223         gpc_j = ra_j / phi_g
224
225         if alpha == 0:
226             rp = ((exppc < z) & (exppc + gpc_j > z)).astype(float)
227         elif alpha == 1:
228             rp = np.where(exppc < z, np.minimum(gpc_j, z - exppc) / z
                , 0.0)
229         else:
230             gap = np.maximum(z - exppc, 0.0)
231             rp = np.where(exppc < z,
                (gap / z) ** alpha - (np.maximum(gap - gpc_j, 0) /
                z) ** alpha,
232             0.0)
233
234     theta = phi_g * np.dot(fw, rp) / fw_sum / ra_j
235

```

```

236         if theta > best_theta:
237             best_theta, best_ra = theta, ra_j
238
239     return best_ra, best_theta
240
241
242 #
=====
243 # 4. TGPR22 -- V1 VECTORISED (NumPy broadcasting, identical result)
244 #
=====
245
246 def tgpr22_vectorised(exppc, fw, phi_g, alpha, z, dcost, npart):
247     """
248     Vectorised version of tgpr22 using NumPy broadcasting.
249     Replaces the inner forvalues loop with a single matrix operation.
250     Result is identical to V0 to machine precision.
251     Speedup: 2.5x - 4.8x depending on alpha.
252     """
253     poor = exppc < z
254     if not poor.any():
255         return 0.0, 0.0
256     ep = exppc[poor]
257     fw_p = fw[poor]
258     gap_p = z - ep
259     max_gap = float(np.max(gap_p))
260     ra_max = min(dcost / phi_g, max_gap)
261     if ra_max <= 0:
262         return 0.0, 0.0
263
264     # Grid vector: ra[j] = j * (ra_max/npart), j = 1..npart-1
265     ra = np.arange(1, npart) * ra_max / npart # shape (npart-1,)
266     gpc = ra / phi_g # shape (npart-1,)
267     fw_sum = fw.sum()
268
269     # Matrix of individual poverty reductions: shape (n_poor, npart
270     # -1)
271     if alpha == 0:
272         R = (ep[:, None] + gpc[None, :]) > z
273         ftgpr = phi_g * (fw_p @ R.astype(float)) / fw_sum
274     elif alpha == 1:
275         R = np.minimum(gpc[None, :], gap_p[:, None]) / z
276         ftgpr = phi_g * (fw_p @ R) / fw_sum
277     else:
278         R = ((gap_p[:, None] / z) ** alpha
279              - (np.maximum(gap_p[:, None] - gpc[None, :], 0) / z) **
280              alpha)
281         ftgpr = phi_g * (fw_p @ R) / fw_sum
282
283     theta = ftgpr / ra
284     idx = int(np.argmax(theta))
285     return ra[idx], theta[idx]
286 #
=====

```

```

287 # 5. GREEDY DATA-GRAPH ALGORITHM (ogtpr main loop)
288 #
=====
289
290 def datagraph(groups, z, alpha, trans, part, tgpr_fn, verbose=False):
291     """
292     Main greedy loop -- replicates ogtpr Stata module.
293     tgpr_fn : tgpr22_original or tgpr22_vectorised
294     Returns  : (cost array, poverty_before, poverty_after)
295     """
296     G = len(groups)
297     cost = np.zeros(G)
298     dcost = trans
299     inc = [g["exppc"].copy() for g in groups]
300
301     def one_sequence(inc, dcost, npart):
302         best_theta, best_i, best_ra = -np.inf, None, 0.0
303         for i, g in enumerate(groups):
304             ra, th = tgpr_fn(inc[i], g["fw"], g["phi_g"],
305                             alpha, z, dcost, npart)
306             if th > best_theta:
307                 best_theta, best_i, best_ra = th, i, ra
308         return best_i, best_ra
309
310     # --- Initial sequence (full budget) ---
311     bi, br = one_sequence(inc, trans, part)
312     if bi is not None:
313         alloc = min(br, dcost)
314         cost[bi] += alloc
315         inc[bi] += alloc / groups[bi]["phi_g"]
316         dcost = trans - cost.sum()
317         if dcost <= trans / 10000:
318             cost[bi] += dcost
319             dcost = 0.0
320
321     h = 1
322     if verbose:
323         print(f"Sequence ...{h}: Remaining p.c. budget "
324               f"{dcost:10.3f} over {trans:10.3f}")
325
326     # --- Subsequent sequences ---
327     while dcost > 0:
328         npart = part if alpha == 0 else max(2, int(dcost / trans *
329                                                    part))
330         bi, br = one_sequence(inc, dcost, npart)
331         if bi is None or br == 0:
332             break
333         alloc = min(br, dcost)
334         cost[bi] += alloc
335         inc[bi] += alloc / groups[bi]["phi_g"]
336         dcost = trans - cost.sum()
337         if dcost <= trans / 10000:
338             cost[bi] += dcost
339             dcost = 0.0
340
341     h += 1
342     if verbose:
343         print(f"Sequence ...{h}: Remaining p.c. budget ")

```

```

341         f"{dcost:10.3f} over {trans:10.3f}")
342
343     p_before = total_fgt([g["exppc"] for g in groups], groups, z,
344                          alpha)
345     p_after  = total_fgt(inc, groups, z, alpha)
346     return cost, p_before, p_after
347
348 #
349 # =====
350 # 6. WATER-FILLING EXACT SOLUTION (Ravallion & Chao 1989, alpha=2)
351 #
352 # =====
353
354 def water_filling(groups, z, trans):
355     """
356     Exact analytical solution for alpha=2 (small-transfer regime).
357     Complexity: O(G log G).
358     Returns (tau array, common level c, n_targeted, validity flag).
359     Validity: tau_g/phi_g < min_gap_g for all targeted groups.
360     """
361     G = len(groups)
362     phi = np.array([g["phi_g"] for g in groups])
363     mu = np.array([g["mu_g"] for g in groups])
364
365     idx = np.argsort(-mu)
366     mu_s = mu[idx]; phi_s = phi[idx]
367
368     cum_phi = np.cumsum(phi_s)
369     cum_phimu = np.cumsum(phi_s * mu_s)
370
371     c = 0.0; k_star = G - 1
372     for k in range(G):
373         c_k = (cum_phimu[k] - trans) / cum_phi[k]
374         if k == G - 1 or mu_s[k + 1] <= c_k:
375             c = max(c_k, 0.0)
376             k_star = k
377             break
378
379     tau = phi * np.maximum(mu - c, 0.0)
380
381     # Check validity condition
382     valid = True
383     for i, g in enumerate(groups):
384         if tau[i] > 0:
385             poor = g["exppc"] < z
386             if poor.any():
387                 min_gap = float(np.min(z - g["exppc"][poor]))
388                 if tau[i] / g["phi_g"] > min_gap:
389                     valid = False
390
391     return tau, c, k_star + 1, valid
392 #
393 # =====

```

```

393 # 7. MAIN: REPLICATE AND BENCHMARK
394 #
=====
395
396 def print_results(groups, z, alpha, trans, cost, p_before, p_after,
397 label, t_ms):
398     print(f"\n {label} (alpha={alpha})")
399     print(f" {'Group':<32} {'PC Transfer':>12} {'GPC Transfer':>13}"
400         )
401     print(" " + "-" * 58)
402     for i, g in enumerate(groups):
403         gpc = cost[i] / g["phi_g"] if cost[i] > 0.01 else 0.0
404         print(f" {g['name']:<32} {cost[i]:>12.3f} {gpc:>13.3f}")
405     rp = p_before - p_after
406     print(f" ',': <58}")
407     print(f" FGT({alpha}) before : {p_before*100:.5f}%")
408     print(f" FGT({alpha}) after : {p_after*100:.5f}%")
409     print(f" Reduction : {rp*100:.5f} pp ({rp/p_before*100:.3f}%
410         relative)")
411     print(f" Time : {t_ms:.1f} ms")
412
413 def main():
414     print("=" * 65)
415     print(" OGTPR REPLICATION CODE -- Araar (2025)")
416     print(" Burkina Faso, Enqu te Prioritaire II (1998)")
417     print("=" * 65)
418
419     # Load data
420     try:
421         groups, N = load_data(DATA_PATH)
422     except FileNotFoundError:
423         sys.exit(f"Data file not found: {DATA_PATH}\n"
424             "Please set DATA_PATH at the top of this script.")
425
426     print(f" N = {N:,} households | Poverty line = {PLINE:,.0f}
427         FCFA "
428         f"| Budget = {TRANS:,.0f} FCFA")
429
430     # Run for each alpha
431
432     for alpha in ALPHA_LIST:
433         print(f"\n{'='*65}")
434         print(f" alpha = {alpha}")
435         print(f"{'='*65}")
436
437         # V0 Original
438         t0 = time.perf_counter()
439         c0, pb0, pa0 = datagraph(groups, z=PLINE, alpha=alpha,
440             trans=TRANS, part=PART,
441             tgpr_fn=tgpr22_original, verbose=
442             True)
443         t0 = (time.perf_counter() - t0) * 1000
444         print_results(groups, PLINE, alpha, TRANS, c0, pb0, pa0, "V0
445             Original", t0)

```

```

440
441 # V1 Vectorised
442 t1 = time.perf_counter()
443 c1, pb1, pa1 = datagraph(groups, z=PLINE, alpha=alpha,
444                          trans=TRANS, part=PART,
445                          tgpr_fn=tgpr22_vectorised)
446 t1 = (time.perf_counter() - t1) * 1000
447 print_results(groups, PLINE, alpha, TRANS, c1, pb1, pa1, "V1
    Vectorised", t1)
448
449 rp0 = pb0 - pa0; rp1 = pb1 - pa1
450 print(f"\n Speedup V1 vs V0 : {t0/t1:.1f}x")
451 print(f" Delta reduction : {abs(rp1-rp0)*100:.8f} pp "
452       f"({'identical' if abs(rp1-rp0)<1e-9 else 'differs'})")
453
454 # Water-filling (alpha=2 only)
455 if alpha == 2:
456     print(f"\n Water-filling (Ravallion & Chao 1989)
457           ")
458     t2 = time.perf_counter()
459     tau_wf, c_wf, k_wf, valid = water_filling(groups, PLINE,
460                                               TRANS)
461     t2 = (time.perf_counter() - t2) * 1000
462     inc_wf = [g["exppc"] + tau_wf[i]/g["phi_g"] if tau_wf[i]
463               ]>0
464               else g["exppc"].copy() for i,g in enumerate(
465                   groups)]
466     p1_wf = total_fgt(inc_wf, groups, PLINE, 2)
467     rp_wf = pb0 - p1_wf
468     print(f" Common level c : {c_wf:,.1f} FCFA")
469     print(f" Groups targeted : {k_wf} / {len(groups)}")
470     print(f" Validity : {'OK (small-transfer)' if
471           valid else 'VIOLATED (large-transfer)'}")
472     print(f" Reduction : {rp_wf*100:.5f} pp ({rp_wf/
473           pb0*100:.3f}%)")
474     print(f" Time : {t2:.4f} ms (speedup vs V0:
475           {t0/t2:,.0f}x)")
476     print(f" DG > WF by : {(rp0-rp_wf)*100:+.5f} pp "
477           f"({'DG better -- large-transfer regime' if rp0>
478             rp_wf else 'WF better'})")
479
480 print(f"\n{'='*65}")
481 print(" BENCHMARK SUMMARY")
482 print(f"\n{'='*65}")
483 print(f" {'alpha':>6} {'V0 (ms)':>10} {'V1 (ms)':>10} {'Speedup
484       ':>9} {'Reduction V0 (pp)':>18}")
485 print(" " + "-" * 56)
486 for alpha in ALPHA_LIST:
487     t0=time.perf_counter(); c0,pb,pa0=datagraph(groups,PLINE,
488         alpha,TRANS,PART,tgpr22_original); t0=(time.perf_counter()
489         -t0)*1000
490     t1=time.perf_counter(); c1,_ ,pa1=datagraph(groups,PLINE,
491         alpha,TRANS,PART,tgpr22_vectorised); t1=(time.perf_counter
492         ()-t1)*1000
493     rp=(pb-pa0)*100
494     print(f" {alpha:>6} {t0:>10.0f} {t1:>10.0f} {t0/t1:>8.1f}x {
495           rp:>18.4f}")
496 print()

```

```
483  
484  
485 if __name__ == "__main__":  
486     main()
```